
DOCUMENTATION
PDF COMES HERE

N Log N Time Algorithms – $O(n \log n)$

The running time grows in proportion to $n \log n$ of the input:

```
for (int i = 1; i < n; i++) {  
    for(int j = 1; j < n; j = j * 2) {  
        System.out.println("Output: " + i + " and " + j);  
    }  
}
```

If n is 8, then this algorithm will run $8 * \log(8) = 8 * 3 = 24$ times.

Polynomial Time Algorithms – $O(n^p)$

These algorithms are even slower than $n \log n$ algorithms.

The term polynomial is a general term which contains quadratic (n^2), cubic (n^3), quartic (n^4), etc. functions.

What's important to know is that **$O(n^2)$** is faster than **$O(n^3)$** which is faster than **$O(n^4)$** , etc.

```
for (int i = 1; i <= n; i++) {  
    for(int j = 1; j <= n; j++) {  
        System.out.println("Output: " + i + " and " + j);  
    }  
}
```

This algorithm will run $8^2 = 64$ times. Note, if we were to nest another for loop, this would become an $O(n^3)$ algorithm.

Exponential Time Algorithms – $O(k^n)$

These algorithms grow in proportion to some factor exponentiated by the input size.

For example, $O(2^n)$ algorithms double with every additional input. So, if $n = 2$, these algorithms will run four times; if $n = 3$, they will run eight times (kind of like the opposite of logarithmic time algorithms).

$O(3^n)$ algorithms triple with every additional input, $O(k^n)$ algorithms will get k times bigger with every additional input.

```
for (int i = 1; i <= Math.pow(2, n); i++){  
    System.out.println("Output: " + i);  
}
```

If n is 8 it will run $2^8 = 256$ times.

Factorial Time Algorithms – O(n!)

```
for (int i = 1; i <= factorial(n); i++){  
    System.out.println("Output: " + i);  
}
```

where factorial(n) simply calculates n!. If n is 8, this algorithm will run $8! = 40320$ times.

Functions ordered by growth rate:

<u>Function</u>	<u>Name</u>
1	Growth is constant
logn	Growth is logarithmic
n	Growth is linear
nlogn	Growth is n-log-n
n²	Growth is quadratic
n³	Growth is cubic
2ⁿ	Growth is exponential
n!	Growth is factorial

- $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$
- To get a feel for how the various functions grow with n, you are advised to study the following figs:

Instance characteristic n							
Time	Name	1	2	4	8	16	32
1	Constant	1	1	1	1	1	1
$\log n$	Logarithmic	0	1	2	3	4	5
n	Linear	1	2	4	8	16	32
$n \log n$	Log linear	0	2	8	24	64	160
n^2	Quadratic	1	4	16	64	256	1024
n^3	Cubic	1	8	64	512	4096	32768
2^n	Exponential	2	4	16	256	65536	4294967296
$n!$	Factorial	1	2	24	40326	20922789888000	26313×10^{33}